You are working on an EXISTING production Next.js project.

Your task is to perform a careful engineering audit, upgrade, hardening, and repair of the existing codebase.

IMPORTANT: This is NOT a redesign or architecture rewrite.

ABSOLUTE PRESERVATION RULE

DO NOT change the existing:

- architecture
- application structure unless required to fix an actual issue
- page layout
- UI design
- colors
- typography
- fonts
- font sizes
- text/content/copy
- spacing
- component appearance
- animations
- responsive design
- navigation structure
- images/icons unless broken
- branding
- visual hierarchy
- user-facing behavior unless it is currently broken

The website should look and feel EXACTLY the same after the work is complete.

Do not "modernize" the UI.
Do not redesign components.
Do not rewrite working components just because you prefer another implementation.
Do not introduce a new framework or architectural pattern.

Focus on backend reliability, security, correctness, maintainability, validation, database safety, payment reliability, dependency health, and production readiness.

WORKFLOW

Do NOT immediately start making broad edits.

Follow this process:

1. EXPLORE
2. AUDIT
3. PLAN
4. IMPLEMENT
5. TEST
6. VERIFY
7. REPORT

Before modifying code, inspect the entire repository enough to understand:

- package.json
- package lockfile
- Next.js configuration
- TypeScript configuration
- environment configuration
- src/app
- src/components
- src/lib
- src/config
- API routes
- middleware
- authentication
- Supabase clients
- repositories/data layer
- Razorpay integration
- webhook handlers
- Shiprocket integration
- database migrations
- RLS policies
- admin authorization
- tests
- linting
- build configuration

Understand the existing architecture before changing anything.

---

1. SECRET / ENVIRONMENT SECURITY

Inspect all environment files and examples.

Pay special attention to:

SUPABASE_SERVICE_ROLE_KEY

A service-role credential must NEVER be committed as an actual secret.

If .env.example contains a real credential:

- replace ONLY the secret value with a safe placeholder
- do not invent a replacement credential
- report clearly that the existing credential must be rotated in Supabase
- check whether other secrets appear committed
- ensure secrets are never exposed to client-side code
- ensure service-role clients remain server-only

Do NOT delete legitimate public environment variables simply because they are public.

Supabase anon/publishable keys can be client-visible when intended and protected by proper RLS.

---

2. ADMIN AUTHENTICATION HARDENING

Inspect the existing admin authentication implementation.

If there is development bypass logic such as:

DEV_BYPASS_ADMIN_AUTH

make it impossible for the bypass to work in production.

The effective rule should require BOTH:

NODE_ENV !== "production"

AND

DEV_BYPASS_ADMIN_AUTH === "true"

Preserve the existing development workflow.

Centralize environment parsing in the project's existing environment/config system where appropriate.

Do not replace the authentication architecture.

---

3. ORDER PERSISTENCE

Inspect checkout/order creation carefully.

Search for temporary implementations such as:

memoryOrders
Date.now() generated order IDs
in-memory order storage
mock order persistence

Production orders must NOT depend on process memory.

Use the project's EXISTING Supabase/database architecture.

Persist:

- order
- authenticated user association where applicable
- order items
- product references
- quantity
- authoritative server-side price
- payment provider order ID
- payment status
- required timestamps

Do NOT trust price values received from the browser.

Continue calculating authoritative prices server-side using database/product information.

Prefer atomic database operations where practical.

Do not redesign the order schema unless absolutely necessary.

If migrations are necessary, make them minimal and backward-compatible.

---

4. USER ORDER AUTHORIZATION

Inspect every endpoint/function that retrieves orders.

A normal authenticated user must ONLY be able to retrieve their own orders.

Never trust a client-provided user ID.

Determine the authenticated user server-side and filter using the authenticated user's ID.

Admin access should continue following the existing admin authorization system.

Test for IDOR-style authorization mistakes.

---

5. RAZORPAY CHECKOUT

Audit the entire Razorpay integration.

Verify:

- amount calculation happens server-side
- currency handling is correct
- product prices cannot be overridden by the browser
- order IDs are validated
- duplicate requests are handled safely
- errors do not expose internal information
- failed DB writes do not create inconsistent payment state
- successful Razorpay order creation can be reconciled with local order persistence

Do NOT replace Razorpay.

Do NOT change the checkout UI.

---

6. RAZORPAY WEBHOOK

Treat payment webhooks as critical financial infrastructure.

Preserve the existing signature verification if correct.

Verify that signature comparison is timing-safe.

Add strict payload validation using the project's existing validation approach or Zod if already available.

Avoid uncontrolled `any` payload handling.

Webhook processing should follow approximately:

receive raw request
→ verify signature
→ parse payload
→ validate expected event structure
→ identify event
→ claim/store event idempotently
→ update corresponding order/payment state

→ mark event processed
→ return success

CRITICAL:

Do NOT silently swallow database failures.

If a critical persistence operation fails, do not falsely acknowledge successful webhook processing.

Allow Razorpay retry behavior where appropriate.

Implement idempotency so duplicate webhook delivery cannot corrupt order/payment state.

Do not weaken signature validation.

---

7. API INPUT VALIDATION

Audit every public API route.

Especially inspect:

- inquiry/contact endpoints
- checkout
- quote requests
- authentication-related routes
- saved product actions
- admin mutations

Validate untrusted input.

Where appropriate use Zod.

Validate things such as:

- required fields
- string length
- email
- phone
- numeric ranges
- quantities
- IDs
- enum values
- object shape

Reject malformed payloads before repository/database operations.

Do not change legitimate frontend forms.

---

8. RATE LIMITING / ABUSE RESISTANCE

Identify publicly writable endpoints.

Examples:

- inquiries
- contact
- quote requests
- checkout initiation

Add reasonable server-side abuse protection where appropriate.

Do not introduce unnecessary infrastructure if a lightweight implementation compatible with the existing deployment architecture is sufficient.

Do not rate-limit Razorpay webhooks in a way that breaks legitimate payment processing.

---

9. ERROR HANDLING

Audit API responses.

Do NOT return raw exception messages, stack traces, database details, secret information, or provider internals to users.

Detailed errors should be logged server-side.

Client responses should be controlled and generic while retaining appropriate HTTP status codes.

Preserve existing user-facing text wherever possible.

---

10. REMOVE UNSAFE PRODUCTION FALLBACKS

Search repositories/data-access modules for:

- memory fallback data
- sample users
- sample orders

- sample inquiries
- sample phone numbers
- fake addresses
- demo records
- hardcoded production fallback responses

Development/demo fallbacks may remain ONLY when explicitly enabled for development/demo environments.

Production database failure must NOT silently cause fake records to appear.

Production should return an appropriate controlled error or empty state depending on the existing application behavior.

Do not remove legitimate test fixtures from test files.

---

11. SUPABASE / DATABASE AUDIT

Inspect all migrations and policies.

Do NOT redesign the database architecture.

Audit every relevant table for:

- RLS enabled where required
- correct SELECT policies
- correct INSERT policies
- correct UPDATE policies
- correct DELETE policies
- user ownership
- admin permissions
- foreign keys
- unique constraints
- useful indexes
- NOT NULL constraints
- sensible validation constraints

Pay particular attention to:

orders
order_items
quotes
contact inquiries
saved products
comparison data

service data
CMS data
webhook events
admin-related tables

Do not loosen RLS simply to make something work.

Fix application/database authorization correctly.

---

12. SUPABASE SERVICE ROLE BOUNDARY

Verify that privileged Supabase clients are server-only.

Never import service-role clients into:

- client components
- browser bundles
- public utilities

Keep privileged operations behind server boundaries.

---

13. SHIPROCKET

Audit the existing Shiprocket integration.

Do NOT replace it.

Check:

- credential handling
- server-only API calls
- token handling
- API error handling
- retries where appropriate
- order/shipment mapping
- malformed response handling
- sensitive logging

Fix reliability problems without changing checkout/order UI.

---

14. HEALTH ENDPOINT

If /api/health exists, ensure it does not expose:

- database errors
- credentials
- connection details
- internal stack traces
- infrastructure information unnecessary to the public

Return minimal health information.

---

15. ENVIRONMENT VALIDATION

Inspect the existing environment configuration.

Validate required production variables.

Production should fail clearly during startup/build/server initialization when genuinely required configuration is missing.

Development-only variables should remain optional where appropriate.

Never expose server-only environment variables through NEXT_PUBLIC_*.

---

16. DEPENDENCIES

Inspect package.json and the lockfile.

Check:

- outdated packages
- deprecated packages
- incompatible versions
- unnecessary packages
- security advisories
- duplicate dependencies

Do NOT blindly upgrade major versions.

Prefer safe patch/minor updates compatible with the existing architecture.

Major upgrades should only happen if clearly necessary and verified.

Do not change Next.js/React architecture merely to use newer packages.

After dependency changes, verify the lockfile is consistent.

---

17. NEXT.JS PRODUCTION HARDENING

Review next.config.ts and related configuration.

Add safe production protections where appropriate, such as security headers, without changing visual behavior.

Consider appropriate headers such as:

- X-Content-Type-Options
- Referrer-Policy
- Permissions-Policy
- frame protection

Be careful with Content-Security-Policy.

Do NOT introduce a CSP that breaks:

- Next.js
- Razorpay
- Supabase
- images
- scripts
- fonts
- existing application functionality

If a correct CSP cannot be safely determined, document the recommendation instead of blindly adding a breaking policy.

---

18. TYPESCRIPT / CODE QUALITY

Run strict static analysis using the project's existing configuration.

Fix genuine issues involving:

- unsafe `any`
- incorrect null handling

- incorrect interfaces
- inconsistent return types
- unhandled promises
- unreachable code
- fragile assumptions
- duplicate backend logic
- unused imports
- dead backend code

Do NOT perform cosmetic refactoring.

Do NOT reformat the entire repository.

Keep diffs focused.

---

19. TESTS

Inspect existing tests before changing behavior.

Add/update tests for critical functionality where practical.

Prioritize:

- admin authorization
- admin bypass production protection
- checkout server-side price calculation
- user order ownership
- API validation
- webhook signature validation
- webhook idempotency
- duplicate webhook delivery
- database failure during webhook processing
- malformed webhook payloads
- production fallback behavior

Tests must verify actual behavior, not merely execute code.

---

20. VERIFICATION

After implementation run, where available:

npm install / npm ci
npm run lint

npm run typecheck
npm test
npm run build

Also inspect package.json for equivalent commands if names differ.

Do not claim something passed unless you actually ran it and observed success.

If a command cannot run because of environment/network/package-registry limitations, explicitly report:

- command attempted
- exact failure category
- whether it appears project-related or environment-related

Do NOT mark unexecuted tests as passed.

VISUAL REGRESSION RULE

After changes, verify that frontend files were not unnecessarily modified.

Review the git diff.

If visual files changed, confirm each change is required for a functional bug.

Do not modify visual code for cleanup purposes.

The following must remain visually unchanged:

- desktop layout
- tablet layout
- mobile layout
- colors
- fonts
- text
- buttons
- cards
- headers
- footers
- navigation
- forms
- spacing
- responsive breakpoints
- animations

If browser tooling is available, compare important pages before and after.

CHANGE DISCIPLINE

Follow these rules throughout the task:

DO:

- make minimal targeted changes
- reuse existing utilities
- reuse existing architecture
- reuse existing database layer
- reuse existing component patterns
- preserve backward compatibility
- inspect before editing
- test after editing
- verify actual behavior

DO NOT:

- redesign
- rewrite the project
- migrate frameworks
- replace Supabase
- replace Razorpay
- replace Shiprocket
- change branding
- change UI
- change text
- change fonts
- change colors
- reorganize folders without necessity
- add unnecessary dependencies
- create duplicate implementations
- suppress TypeScript errors using broad casts
- disable ESLint rules merely to make checks pass
- weaken RLS
- expose service-role credentials
- trust client-supplied prices or user IDs
- claim verification without actually running it

IMPORTANT: DO NOT OVER-ENGINEER

Prefer the smallest production-safe fix.

If something is already correct, leave it alone.

Do not create helper files, abstractions, services, hooks, wrappers, or utilities unless they solve a real issue.

Avoid changing 20 files when 3 files can correctly solve the problem.

EXECUTION

First inspect the repository and produce a concise internal plan.

Then implement fixes in logical groups.

After each critical group, run relevant verification.

Before finishing:

1. Review the complete git diff.
2. Remove accidental/unnecessary changes.
3. Confirm no design/UI changes occurred.
4. Confirm no secrets were introduced.
5. Run final lint/typecheck/tests/build where available.

FINAL REPORT

When finished, provide a concise report containing:

Critical fixes

What critical problems were fixed.

Security

Authentication, secrets, validation, RLS and API hardening completed.

Payments

Razorpay checkout/webhook improvements.

Database

Persistence, policies, indexes or constraints changed.

Reliability

Error handling, fallback behavior and Shiprocket improvements.

Dependencies

Exactly which dependencies changed and why.

Tests

Tests added/updated and their results.

Verification

Report the ACTUAL results of:

- lint
- typecheck
- tests
- production build

Files changed

List every modified/created migration/config/source file with a one-line reason.

Manual actions required

Clearly identify actions I must perform myself, especially:

- rotating compromised Supabase service-role credentials
- adding/replacing deployment environment variables
- applying new Supabase migrations
- configuring external provider settings

UI preservation

Explicitly confirm whether any user-facing layout, design, colors, fonts, text, spacing, or branding changed.

Do not say "everything is fixed" unless verification proves it.

Start by exploring the repository. Do not redesign anything.